



Universidade do Minho
Licenciatura em Engenharia Informática

Relatório da Fase 2- Laboratórios de Informática III

Ano Lectivo de 2025/2026

Grupo 24

Sara Marques a111182;

Marta Araújo a110317;

Teresa Teixeira a111415.

1. Introdução	1
2. Sistema	2
2.1 Diagrama da Arquitetura	2
2.2 Módulos Principais	3
2.3 Justificação das Opções de Implementação	3
3. Estruturas de Dados e Algoritmos por Query	3
3.1 Query 1: Resumo de Aeroporto	4
3.2 Query 2: Top N Aeronaves (Agregação)	4
3.3. Query 3: Aeroporto com Mais Partidas	4
3.4. Query 4: Passageiro Mais Frequente no Top 10 (Agrupamento Semanal)	5
3.5. Query 5: Top N Companhias com Atraso (Array Estático)	6
3.6. Query 6: Aeroporto de Destino Mais Comum	6
3.7 Modularidade e Encapsulamento	7
4. Interface Interativa	7
5. Discussão	7
5.1 Análise de Desempenho	7
6. Conclusão	10

1. Introdução

Este relatório descreve o desenvolvimento e execução do trabalho prático realizado no âmbito da unidade curricular Laboratórios de Informática III, cujo objetivo central consiste na criação e implementação de um sistema para o processamento, armazenamento e análise de grandes volumes de dados de tráfego aéreo e a execução de um conjunto de interrogações (queries) definidas sobre essa informação.

O sistema apresentado e implementado foi realizado com uma arquitetura modular, que assegura uma separação clara de responsabilidades entre os seus diferentes componentes, nomeadamente o carregamento e validação dos dados, a sua organização em estruturas de dados apropriadas e a execução de interrogações (queries) com formatos de saída distintos. Esta abordagem contribui para uma maior legibilidade, facilitando a introdução de novas funcionalidades e otimizações ao longo do desenvolvimento do sistema.

Ao longo do trabalho, foram exploradas várias estratégias de implementação e escolhidas as estruturas de dados mais apropriadas para cada interrogação (query), tendo em conta a eficiência. Na fase final, foi realizada uma análise de desempenho em detalhe, com recurso a testes em diferentes ambientes, de modo a avaliar o impacto das decisões tomadas e identificar possíveis limitações da solução desenvolvida.

Deste modo, o relatório apresentado fornece uma visão abrangente do sistema desenvolvido, referindo os aspetos mais significativos e as principais decisões de projeto que orientaram a sua implementação e conceção.

Um dos grandes desafios nesta segunda fase residiu na gestão do compromisso entre o tempo de carregamento (parsing) e o tempo de resposta às interrogações (queries). Dado que o sistema deve responder a múltiplos pedidos num curto espaço de tempo, a estratégia adotada privilegiou o pré-processamento. Isto significa que a complexidade computacional foi, sempre que possível, antecipada para a fase de leitura dos ficheiros, de forma a estruturas de dados auxiliares (índices) que conduzem a respostas quase instantâneas. Este relatório detalha não só essa arquitetura, mas também a nova interface interativa baseada em ncurses.

2. Sistema

O sistema foi desenhado no seguimento de uma arquitetura modular e estruturada em camadas, de forma a priorizar o encapsulamento de dados, robustez na validação de inputs e eficiência algorítmica no acesso à informação. Utilizamos a linguagem C e recorremos à biblioteca GLib para gestão de estruturas de dados complexas, garantindo portabilidade e gestão otimizada de memória.

2.1 Diagrama da Arquitetura

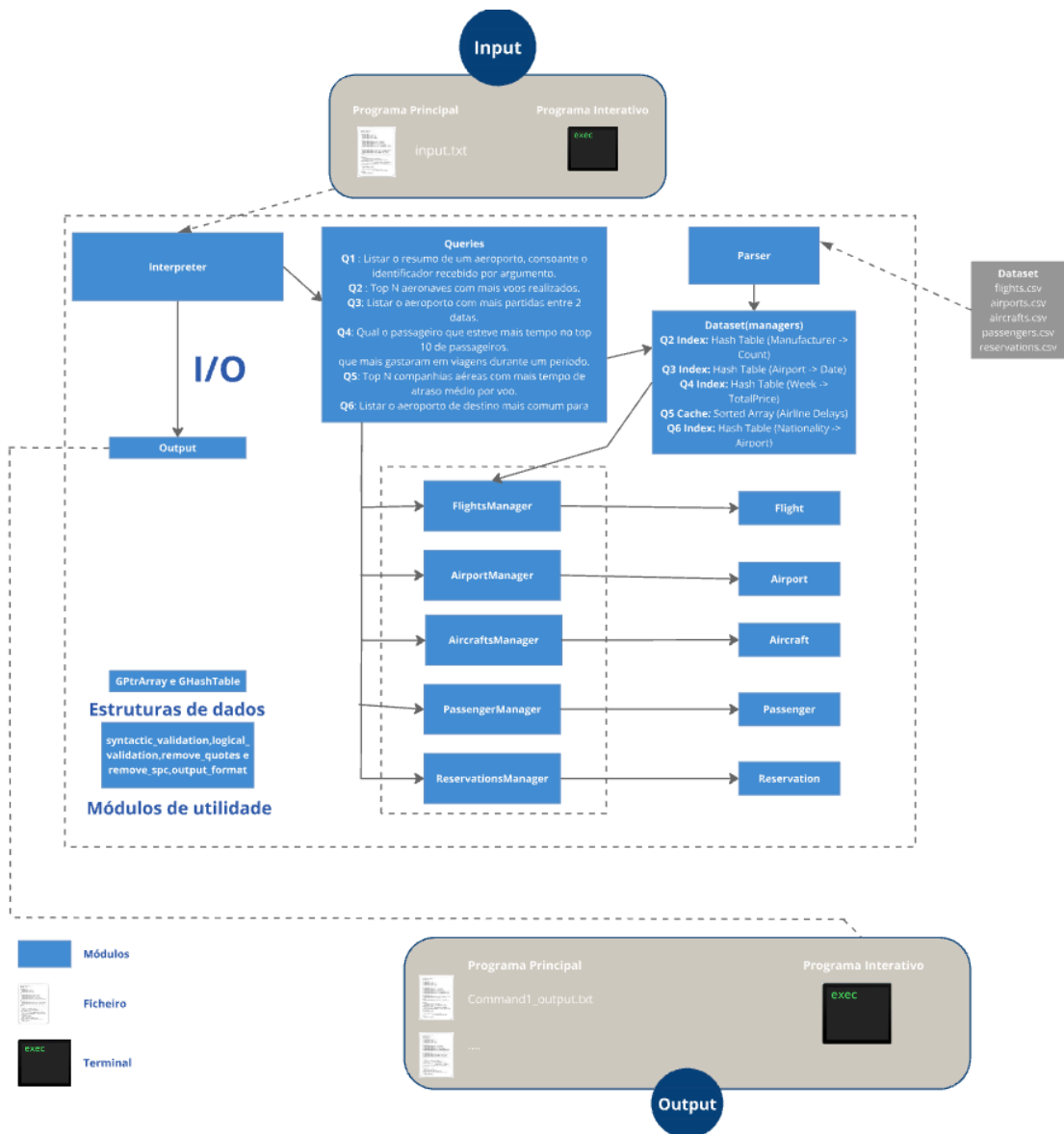


Figura 1 - Ilustração da arquitetura do sistema.

2.2 Módulos Principais

O fluxo de dados inicia-se no módulo de entrada, que suporta dois modos de execução distintos: um modo *batch*, para processamento sequencial de ficheiros de comandos, e um modo interativo via terminal.

- **Parser e Validação:** A entrada de dados é assegurada pelo componente Parser, responsável pela leitura dos ficheiros CSV e pela aplicação de validação rigorosa (sintática e lógica).
- **Dataset e Managers:** Centralmente, encontramos o Dataset, que agrega os gestores de entidades (*Managers*). Os dados principais são armazenados em *Hash Tables* para permitir acessos individuais com complexidade média de $O(1)$.
- **Motor de Queries:** O processamento de pedidos é gerido por um módulo interpretador que respeita o encapsulamento e encaminha os comandos para as respetivas funções, formatando a saída conforme necessário.

2.3 Justificação das Opções de Implementação

- **Pré-computação (Índices):** Optou-se por calcular estatísticas pesadas durante o carregamento dos dados. Embora aumente ligeiramente o tempo de arranque (*startup*) e o consumo de memória, permite que a execução das queries seja instantânea.
- **Encapsulamento via Ponteiros Opacos:** A utilização de *Opaque Pointers* (ex: `typedef struct dataset *Dataset;`) nos ficheiros de cabeçalho impede que módulos externos acedam diretamente aos membros das estruturas, facilitando a manutenção.
- **Uso da GLib:** A escolha da GLib justifica-se pela robustez e eficiência das suas implementações de *Hash Tables*, evitando erros comuns de gestão de memória.

3. Estruturas de Dados e Algoritmos por Query

Nesta secção, detalha-se a implementação específica de cada uma das seis interrogações, justificando as estruturas de dados auxiliares criadas para garantir a eficiência exigida.

3.1 Query 1: Resumo de Aeroporto

Objetivo: Listar o resumo de um aeroporto, incluindo o total de voos (partidas/chegadas), passageiros e companhias distintas.

- **Estratégia:** Otimização via Acumulação no Parsing.
- **Estrutura de Dados:** A estrutura interna do `Airport` foi enriquecida com contadores atômicos (`num_departures`, `num_arrivals`, etc.). Não existe uma estrutura de dados separada para esta query; utiliza-se o próprio `AirportsManager`.
- **Algoritmo:**
 - Durante a leitura do ficheiro `flights.csv`, ao validar um voo, o sistema acede ao aeroporto de origem e incrementa o contador de partidas. Faz o mesmo para o destino (chegadas).
 - O mesmo processo ocorre para os passageiros durante a leitura de `reservations.csv`.
- **Execução:**
 - A query limita-se a fazer um *lookup* do aeroporto pelo código IATA e devolver os valores já calculados. Não há iteração sobre voos no momento da execução.

3.2 Query 2: Top N Aeronaves (Agregação)

Objetivo: Listar os voos de um fabricante (ou todos), agrupados por modelo.

- **Estrutura de Dados:** Tabela de Dispersão Aninhada (Nested Hash Table). Implementou-se no `Dataset` um índice auxiliar do tipo: `GHashTable<Fabricante, GHashTable<Modelo, Contagem>>`. Existe também uma chave especial reservada, `__ALL__`, que agrega todos os fabricantes.
- **Algoritmo:**
 1. Ao ler um voo válido, o sistema identifica a aeronave (`aircraft_id`) e, consequentemente, o seu fabricante e modelo.
 2. Acede à tabela externa usando a chave "Fabricante".
 3. Acede à tabela interna e incrementa o contador associado ao "Modelo".

3.3. Query 3: Aeroporto com Mais Partidas

Objetivo: Identificar o aeroporto com o maior número de partidas num determinado intervalo de datas, considerando apenas voos efetivamente realizados (não cancelados).

- **Estrutura de Dados:** `GHashTable *q3_index`: tabela hash pré-computada que associa cada aeroporto a outra tabela hash de datas e contadores de partidas (`date -> count`). `GHashTable *airport_totals`: tabela hash usada temporariamente para acumular o total de partidas por aeroporto dentro do intervalo de datas especificado. `AirportCount`: estrutura auxiliar para representar o código do aeroporto e o total de partidas (usada de forma conceitual, não explicitamente no código).

- **Algoritmo:**
 1. Obtém o índice pré-computado do dataset (`q3_index`) e cria a tabela `airport_totals` para armazenar os totais de partidas por aeroporto.
 2. Iteração sobre aeroportos: percorre todos os aeroportos presentes no índice: Para cada aeroporto, itera sobre suas datas registradas no índice. Para cada data, verifica se está dentro do intervalo `[start_date, end_date]`. Se estiver, soma o número de partidas no contador total do aeroporto. Se o aeroporto tiver pelo menos uma partida no intervalo, armazena o total na tabela `airport_totals`.
 3. Determinação do aeroporto com mais partidas: percorre `airport_totals` para encontrar o aeroporto com o maior número de partidas. Em caso de empate, escolhe o aeroporto com código lexicograficamente menor.
- **Execução:** A query percorre linearmente todos os aeroportos e, para cada aeroporto, todas as datas registradas.

3.4. Query 4: Passageiro Mais Frequente no Top 10 (Agrupamento Semanal)

Objetivo: Identificar o passageiro que aparece mais vezes no Top 10 de gastos semanais, num dado intervalo de datas (opcional). Cada semana é considerada de domingo a sábado, e apenas passageiros com registros de gastos em voos são contabilizados.

Estrutura de Dados: `GHashTable *q4_weeks`: tabela hash do dataset, onde cada chave representa o início da semana (YYYY-MM-DD) e o valor é outra tabela hash com gastos dos passageiros nessa semana. `GHashTable *top10_counts`: tabela hash para contar quantas vezes cada passageiro aparece no Top 10 semanal. **PassengerSpending**: estrutura contendo o ID do passageiro (`document_number`) e o total gasto naquela semana (`total_spent`). **FilterContext**: estrutura que mantém informações sobre o uso do filtro de datas e a tabela de contagem `top10_counts`.

- **Algoritmo:**
 1. Verifica se foi fornecido um intervalo de datas; caso positivo, ajusta o intervalo para semanas completas (domingo a sábado).
 2. Para cada semana em `q4_weeks`: Aplica o filtro de datas, se ativo. Converte a tabela de gastos da semana em uma lista de `PassengerSpending`. Ordena a lista decrescentemente pelo total gasto; desempata lexicograficamente pelo ID do passageiro.
 3. Seleciona os Top 10 passageiros da semana e incrementa sua contagem na tabela `top10_counts`.
 4. Após processar todas as semanas, identifica o passageiro com mais aparições no Top 10; em caso de empate, escolhe o ID menor lexicograficamente.
- **Execução:**
 1. A query percorre todas as semanas do dataset e, para cada semana, processa os passageiros que tiveram gastos registrados.
 2. Ela cria uma lista ordenada de passageiros com base no total gasto e seleciona os Top 10 da semana. Em seguida, conta quantas vezes cada passageiro aparece no Top 10 ao longo de todas as semanas consideradas.

3. Ao final, identifica o passageiro com mais aparições, consultando seus dados completos para produzir a saída final. Caso não haja passageiros ou o filtro de datas exclua todos os registros, a query retorna uma saída vazia.

3.5. Query 5: Top N Companhias com Atraso (Array Estático)

Objetivo: Listar companhias aéreas ordenadas pelo atraso médio dos voos correspondentes.

- **Estratégia:** Conversão para Estrutura Estática. Dado que os dados históricos não mudam após o carregamento, não é eficiente usar estruturas dinâmicas ou reordenar dados a cada pedido.
- **Estrutura de Dados:**
 1. *Durante o Parsing:* GHashTable temporária para acumular `total_delay` e `flight_count` por companhia.
 2. *Pós-Parsing:* Um **Array Estático de Structs** (`Q5Stats[ ]`), já ordenado.

3.6. Query 6: Aeroporto de Destino Mais Comum

Objetivo: Determinar os N aeroportos de destino mais populares para uma dada nacionalidade.

- **Desafio:** Relacionamento entre 3 entidades (Passageiro -> Reserva -> Voo).
- **Estrutura de Dados:** Índice de Nacionalidades. `GHashTable<Nacionalidade, GHashTable<Aeroporto, Contagem>>`
- **Algoritmo:**
 1. Ao processar uma reserva, o sistema verifica a nacionalidade do passageiro.
 2. Percorre a lista de voos associada à reserva.
 3. Para cada voo, incrementa o contador do aeroporto de destino na entrada daquela nacionalidade.
- **Execução:** Acesso direto à tabela da nacionalidade pedida, seguido de ordenação dos aeroportos por contagem.

3.7 Modularidade e Encapsulamento

O projeto adota rigorosamente o conceito de Tipos Abstratos de Dados (TADs) através de Ponteiros Opacos. As estruturas de dados (como `struct dataset`) são definidas apenas nos ficheiros de implementação (.c), sendo expostas nos cabeçalhos (.h) apenas como ponteiros (`typedef struct dataset *Dataset;`).

Esta arquitetura impede que os módulos clientes (Queries e Interface) acedam diretamente aos campos internos, obrigando ao uso da API pública (*Getters*). Isto permitiu, por exemplo, alterar radicalmente a implementação interna da Query 5 (de lista para array) sem quebrar o resto do código, demonstrando a elevada manutenção do sistema.

4. Interface Interativa

Para além do modo *batch*, foi desenvolvido um modo interativo que utiliza a biblioteca *ncurses*. Esta biblioteca foi escolhida dada a sua possibilidade de proporcionar uma experiência de utilizador superior. No programa interativo são abordados os seguintes pontos:

- **Navegação:** Menu controlável por setas e atalhos de teclado.
- **Paginação:** O sistema de output suporta *scroll*, permitindo visualizar queries com milhares de resultados sem "inundar" o terminal.
- **Inputs Dinâmicos:** A interface adapta-se à query seleccionada, pedindo apenas os argumentos necessários (ex: datas, IDs) e validando-os antes da execução.

5. Discussão

Nesta secção, analisa-se criticamente as opções de implementação, as estruturas de dados seleccionadas e o desempenho obtido.

5.1 Análise de Desempenho

A metodologia de testes baseou-se na execução do programa-testes, recorrendo exclusivamente ao *dataset large*, de modo a avaliar o comportamento do sistema sob um grande volume de dados. O objetivo principal foi minimizar o tempo de CPU na execução das queries, através da substituição de operações lineares sobre o histórico de voos por acessos diretos a estruturas previamente indexadas ou por iterações de dimensão reduzida.

Tabela 1: Características das diferentes máquinas

Características:	Computador 1	Computador 2	Computador 3
Modelo Hardware	Lenovo Yoga Pro 7 14IMH9	Macbook Pro	MSI Cyborg 15 A13V
Memória	32.0 GiB	8.0 GiB	16.0GiB
Processador	Intel® Core™ Ultra 7 155H × 22	Apple M3	13th Gen Intel® Core™ i5-13420H x 12
Gráficos	Intel® Arc™ Graphics (MTL)	Apple M3	Intel® Graphics (RPL-P)
Capacidade disco	1.0 TB	494,38 GB	512.1 GB
Sistema operativo	Ubuntu 24.04.2 LTS	MacOS	Ubuntu 24.04.1 LTS
Versão Kernel	Linux 6.14.0-33-generic	25.0.0	Linux 6.14.0-35-generic

Tabela 2: Tempos das diferentes queries nas diferentes máquinas:

Tempos	Computador 1	Computador 2	Computador 3
Query 1	10 µs 8 µs 5 µs	179 µs 56 µs 40 µs	5 µs 6 µs 7 µs
Query 2	11 µs 9 µs 6 µs	79 µs 47 µs 40 µs	9 µs 7 µs 6 µs
Query 3	9 µs 8 µs 6 µs	128 µs 41 µs 34 µs	7 µs 4 µs 9 µs
Query 4	7 µs 7 µs 5 µs	86 µs 38 µs 33 µs	9 µs 8 µs 6 µs
Query 5	6 µs 8 µs 6 µs	67 µs 36 µs 34 µs	7 µs 7 µs 6 µs
Query 6	7 µs 7 µs 5 µs	90 µs 30 µs 29 µs	5 µs 6 µs 7 µs

Tabela 3: Média dos tempos das diferentes queries nas diferentes máquinas:

Médias dos tempos	Computador 1	Computador 2	Computador 3
Query 1	7.66 μ s	91.7 μ s	6 μ s
Query 2	8.66 μ s	55.3 μ s	7.3 μ s
Query 3	7.66 μ s	67.6 μ s	6.7 μ s
Query 4	6.33 μ s	52.3 μ s	7.7 μ s
Query 5	6.66 μ s	45.6 μ s	6.7 μ s
Query 6	6.33 μ s	49.6 μ s	6 μ s

Os tempos médios de execução obtidos, apresentados na Tabela 3, situam-se consistentemente na ordem dos microsegundos para todas as queries, o que evidencia a adequação da arquitetura adotada para volumes elevados de dados. Estes resultados são justificados pela utilização sistemática de estruturas de dados baseadas em indexação, nomeadamente *hash tables*, que permitem acessos diretos e evitam percursos lineares sobre o dataset durante a fase de execução das queries. A maior parte do custo computacional é transferida para a fase inicial de carregamento e pré-processamento dos dados, reduzindo significativamente o tempo necessário para responder a cada consulta. Se analisarmos comparativamente as 3 máquinas, há variações nos tempos médios, atribuíveis a diferenças de hardware e ambiente de execução. Estas variações mantêm uma tendência consistente, confirmando que o desempenho do sistema é estável. Com estes resultados confirmamos que a solução implementada responde de forma eficiente a operações de consulta complexas, mesmo quando aplicada a datasets de grande dimensão.

6. Conclusão

A realização da 2.^a Fase do trabalho prático permitiu-nos mudar de uma abordagem focada apenas na correção funcional, para uma perspectiva orientada à otimização, escalabilidade e robustez.

Ao longo do desenvolvimento, a principal lição aprendida foi a importância crítica da escolha das estruturas de dados. Ficou evidente que, para sistemas de grande escala, a eficiência não se alcança apenas com código limpo, mas sim através de estratégias arquiteturais, como a indexação prévia (pré-computação). A decisão de transferir a complexidade algorítmica para a fase de carregamento (parsing) provou ser a estratégia correta para garantir a reatividade do sistema.

Como melhorias futuras, identifica-se o carregamento paralelo como prioridade. Dado que a arquitetura atual já garante a independência entre Managers, seria possível utilizar multithreading para processar os ficheiros CSV (Voos, Reservas, Passageiros) simultaneamente, reduzindo drasticamente o tempo de arranque do sistema em processadores multi-core.